

An application of constraint solving for home health care

Massimiliano Cattafi ^a, Rosa Herrero ^b, Marco Gavanelli ^{c,*}, Maddalena Nonato ^c and Federico Malucelli ^d

^a *Department of Electrical and Electronic Engineering, Imperial College London, London, UK*
E-mail: m.cattafi@imperial.ac.uk

^b *Department de Telecomunicació i Enginyeria de Sistemes, Universitat Autònoma de Barcelona, Cerdanyola del Vallès, Spain*

E-mail: RHerrero.Math@gmail.com

^c *EnDIF, Università di Ferrara, Ferrara, Italy*

E-mails: marco.gavanelli@unife.it, maddalena.nonato@unife.it

^d *Dipartimento di Elettronica e Informazione, Politecnico di Milano, Milano, Italy*

E-mail: malucell@elet.polimi.it

Abstract. Although sometimes it is necessary, no one likes to stay in a hospital, and patients who need to stay in bed but do not require constant medical surveillance prefer their own bed at home. At the same time, a patient in a hospital has a high cost for the community, that is not acceptable if the patient needs service only a few minutes a day. For these reasons, the current trend in Europe and North-America is to send nurses to visit patients at their home: this choice reduces costs for the community and gives better quality of life to patients. The challenge is to deliver the service in a cost effective manner without a detriment of the service quality. These social and health management issues have interesting implications from the mathematical viewpoint, introducing a challenging combinatorial optimization problem. The problem consists in assigning patients' services to traveling nurses and defining the nurse itineraries so that the following optimization aspects are considered: the nurse workloads (including service as well as travel time) are balanced, patients are preferentially served by a single nurse or just a few ones, and the overall travel time is minimized. These objectives are somehow conflicting and a reasonable trade off must be found. The complexity of the problem calls for suitable optimization-based algorithmic support to decisions, in particular in the perspective of an increasing diffusion of the service.

This problem is known in the literature as the Home Health Care (HHC) problem. In this paper, we address the HHC problem in the municipality of Ferrara, a mid-sized city in the North of Italy. The problem is currently solved by hand, starting from a partitioning of patients based on predefined zones. We describe a Constraint Programming model that solves the HHC problem, and show significant improvements with respect to the current manual solution.

Keywords: Constraint programming, nurse scheduling applications, home health care, large neighbourhood search, routing problems, Lagrangian relaxation

1. Introduction

In recent years, the Health Services providers in Europe and North America have tried to avoid the hospitalization of patients whenever possible. A patient that has to stay in a hospital has a high cost for the community, and saving money (and resources) means that

those resources can be spent for providing a better service to other patients. Moreover, patients perceive a higher quality of life when they stay at home, with their dear ones, and feel their illness more similar to a “normal” life situation. This reduces depression risk for the patient, and improves rehabilitation rate. Indeed, high quality home health care following hospital dismissal has proved essential in reducing hospital readmissions, if able to cope with preventable complications. Therefore, it is crucial to deliver the service in a cost effective manner while not deteriorating service quality. Arguments in favor of home health care and how to re-

* Corresponding author: Marco Gavanelli, EnDIF (Engineering Department in Ferrara), Università di Ferrara, Via G. Saragat, 1, 44122 Ferrara, Italy. Tel.: +39 0532 974833; Fax: +39 0532 974833; E-mail: marco.gavanelli@unife.it.

duce health related risks when clinical service is provided at home can be found also in [21] and [39]. When a patient requires service only for a limited time per day, or even per week, then it is a viable and convenient solution to send nurses to provide the service at the patient's home. Managing such a service gives rise to a family of challenging optimization problems referred to as home health care routing and scheduling (see [5] and [57]). In this paper we describe an application concerning home health care in the city of Ferrara, Italy, and how it can be approached by hybrid Constraint Programming (CP) and Lagrangian Relaxation. A previous version of this work was also presented in the technical communication [11]. The current article extends [11] in several directions (that we will detail better in the rest of the paper):

- a new implementation in the language Comet, while the previous was in the logic language ECLⁱPS^e [52]. Comet has more global constraints, that have powerful constraint propagation, and allowed us to improve the efficiency of the application;
- new search algorithms, based on Large Neighbourhood Search, that let us get to better solutions in shorter time;
- a wider experimentation;
- the use of a new objective function.

1.1. The home health care service in Ferrara

At present, the home health care (HHC) service in the city of Ferrara, Italy, is directly administrated by the local agency of the National Health Service (NHS), namely AUSL 109. All patients who are not self sufficient and in need of medical treatment are eligible for HHC, i.e., any person who is infirm, chronically ill, or disabled. A demand for assistance must be issued by the family doctor to AUSL 109, specifying the list of required medical treatments. If the demand is accepted, the patient is enrolled in the service, and the required treatments are entered into the request data base. Each request is characterized by a patient identifier (name and address), a medical treatment, and the specific day of the week when the treatment must be delivered (each patient can have more than one request per week).

Service is provided by a set of qualified nurses, all registered nurses with additional training, employed by the local NHS agency. Every day, each nurse who is on duty starts the shift at the city hospital, visits the patients in the list delivering the required treatments

and traveling by car from one patient's home to the next; finally the nurse returns to the hospital.

A total of 15 nurses is involved and they are organized as follows. On working days, i.e., from Monday to Friday, 12 nurses are on duty. 9 of them operate in the morning while 3 in the afternoon. During weekends, 4 nurses work on Saturday and 1 on Sunday or during holidays.

Working rules. A duty should last up to 7 hours and 12 minutes on a working day, and about 6 hours on Saturdays and holidays. This limit is not strictly enforced, provided that the threshold of 36 hours per week is not exceeded over a planning horizon of 4 weeks.

Each nurse works 5 days a week (on working days only) each alternate week and works 6 days the other week; the sixth day is usually a Saturday, but it can be a Sunday once every 14 weeks.

Lunch breaks are not present because shifts are either morning or afternoon. All nurses have same starting and ending times, and must start and end shifts at the hospital. Nurse unavailabilities are considered unexpected events, and one additional nurse is always available for such emergency.

Services consist of treatments and medications, and since their delivery time has almost no impact on the patient daily routine (patients stay in bed almost all day), there are no time windows. A treatment lasts from 5 to 60 minutes, depending on its specific characteristics (see Table 1 for an excerpt). If several treatments are assigned to the same patient on the same day, these treatments are carried out as a whole by a single nurse and handled as a single request. Therefore, requests duration is quite heterogeneous over the whole set of requests, as can be seen on Fig. 1.

Ferrara has about 150,000 inhabitants, and the area administered by AUSL 109 is rather large and its population ageing. Most of the elderly patients live in the countryside, so there is a significant geographical dis-

Table 1
Excerpt of the services with average service time

Service	Time
First visit	1 hour
Enema	30 min
General check	15 min
Tracheotomy check and tube change	15 min
Central venous catheter	30 min
Electrocardiogram	30 min
Emogasanalysis	15 min
Mouth cleaning	5 min

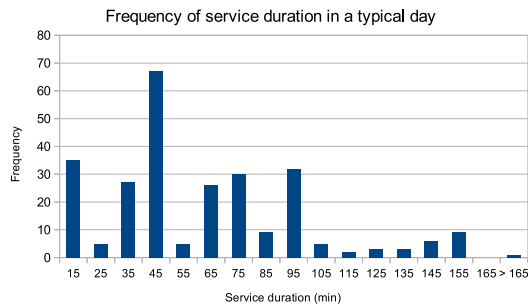


Fig. 1. Histogram of the frequency of service durations in a typical day. (Colors are visible in the online version of the article; <http://dx.doi.org/10.3233/AIC-140632>.)

person of the requests. Therefore, the service is characterized by a high variance of duration and a significant geographical dispersion of the requests.

As a sample of the set of requests, we consider the month of February 2010, with 3323 requests subdivided among 458 patients. Several patients are located either in town or in its suburbs, though quite a few live in the neighboring towns of Masi Torello and Voghiera, that are approximately 20 km away.

At present, the digitization of data is limited to the patient data base, containing the address and the prescribed services for each patient. The duty definition and the management of nurse agendas is handled manually and supervised by the chief nurse, with frequent adjustments done by cell phone. As a record of their duty, nurses fill forms on paper which they return to the chief nurse at the end of their daily shift.

1.2. The problem data

Optimizing the health home care scheduling is essential to make the service cost-effective and to avoid the so called burn out phenomenon, i.e. nurses who get tired and act in an unfriendly manner to patients or even leave the job. At the same time, also the patient point of view should be accounted for. Specifically, a patient would like to receive cares always by the same nurse or by a few ones, if possible, since receiving clinical care involves physical and emotional contacts that one is not like to easily share with anyone. This introduces the concept of *loyalty* (i.e., the number of different nurses per patient) as a measure of the service quality as it is perceived from the patient viewpoint.

We start by examining the current solution approach, as it is manually carried out by the people in AUSL 109, and analysing its critical aspects.

Nurses organize their duties themselves, solving the problem by hand according to the following procedure.

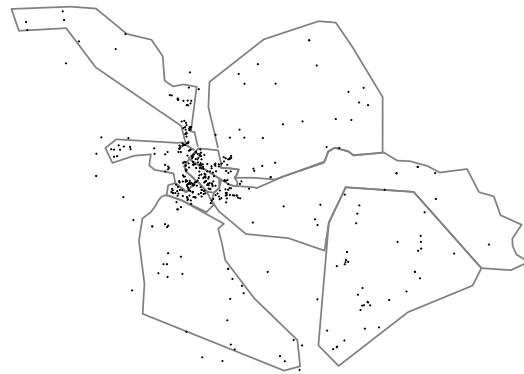


Fig. 2. The 9 zones in which the city is divided, with shown the location of the patients.

To simplify the assignment of patients to nurses, the territory pertaining AUSL 109 is statically partitioned into 9 zones (see Fig. 2), considering factors like the distribution of the population, its age and historical data. As we can see from Fig. 2, some zones in the city center are very small but have a large number of patients. Other zones in peripheral areas are larger but less densely populated. A nurse assigned to a central zone will probably have many patients to serve, while a nurse assigned to a peripheral zone will likely spend a significant amount of time traveling.

Each nurse receives in charge the services of all the patients belonging to one area. Then the nurse tries to fit the requests for the whole week into the working shifts, while complying with the maximum weekly workload allowed, since no overtime is permitted. The assignment of requests to daily shifts follows a sort of greedy criterion: nurses, by hand, insert as many requests as possible into the shifts. There are no given rules on which patients to select first: each nurse could use a different order, in principle. Such decisions are not driven by any optimization criteria, and the routing is not necessarily optimal.

If a nurse can not fulfill all the requests, some requests are forwarded to the chief nurse who is in charge of the service coordination and reassigns them to other colleagues. In such a case, a patient may be treated by different nurses at different days of the week, which goes to the detriment of the service quality. The three nurses working in the afternoon are in charge of the remaining requests, that are still assigned to nurses on the basis of a similar geographical partition of the territory, each nurse dealing with the patients of three zones.

Due to this greedy procedure, the nurse weekly schedules have very different workloads, and balancing this over the months leads to a detriment in loyalty.

Nurses complain about such disparities, and have difficulties adapting their schedule to new patients, new treatments, or to any other change. Moreover, if the workload balance could be improved by optimizing the routing component, nurses could be available at the hospital for other tasks. In addition, an improved routing plan would impact on the direct expenses related to fuel and car usage, which contribute to the overall cost.

The decisions to be taken are:

- partitioning the requests to a set of duties,
- for each duty determining the sequence of services to be carried out optimizing the traveling times by solving a traveling salesman problem (TSP), and
- assigning each duty to a nurse to build the nurse weekly schedule.

Scheduling such a service poses several challenges related to the criteria which characterize high quality solutions. A good solution should achieve at the same time:

- from the point of view of the NHS, the minimization of the travel time over the service time, since during travel a nurse is on duty but is not delivering any service;
- from the point of view of the nurses, a fair distribution of the workload, which can not be guaranteed by simply equally subdividing patients, due to heterogeneous requests;
- from the point of view of the patients, a high degree of *loyalty*, i.e., the number of different nurses who are in charge of a single patient should be minimal.

The problem we are facing contains several sub-problems, namely assigning each request to one available nurse, scheduling optimally the services in each nurse's duty, and finding the optimal route for each nurse in order to avoid long useless trips. We will describe in the following the main components of the CP-based hybrid solution approach that has been developed to tackle such a difficult problem.

2. A Constraint Program

We believe that using Constraint Programming (CP) to tackle this problem is a wise choice, because of the importance of the feasibility issues represented by the time constraints, of the many aspects which can benefit of CP's flexibility (such as the need to easily add and

relax constraints to follow change in the requirements) and because CP provides an effective framework for the integration of components based on different approaches. In this section we will explain the CP model and we will deal with some of these issues.

2.1. Input description

The input data consists of:

- a set S_{serv} of services, of size N_s ; for each service s we know the patient pat_s , the day day_s and the duration dur_s ;
- a matrix of distances D where the element $d_{i,j}$ is the travel time from patient i to patient j ;
- the number of nurses N_n ;
- the set of nurses S_{nurse} (i.e., $N_n = |S_{nurse}|$ or $S_{nurse} = \{1, \dots, N_n\}$);
- the number of days N_d considered in the scheduling;
- the maximum time that a nurse can spend working each day; $MaxDayWL$ is the amount of minutes available per day for each nurse (including service time and travel time); it currently corresponds to 7 hours and 12 minutes (432 minutes);
- the maximum time that a nurse can spend working in a week; $MaxWeekWL$ is the number of minutes available per week for each nurse. It currently corresponds to 36 hours.

2.2. The CP model

A constraint model is defined by a set of decision variables (unknowns), each ranging on a given domain, and a set of constraints that each assignment must satisfy. Often, there is also an objective function that should be minimized or maximized. A solution is an assignment of values taken from the domains to the corresponding variables such that all constraints are satisfied, and that minimizes (resp. maximizes) the objective function.

Usually an efficient model makes use of *global constraints*, i.e., constraints that involve many (possibly, all) variables in the model. Often, global constraints propagate more efficiently than small constraints that involve only two or three variables. We show a model that includes a number of global constraints.

2.2.1. Variables

The main decision is assigning nurses to services. To each service s we associate a decision variable $Nurse_s$ with domain $\mathcal{S}_{nurse}$.

Other variables have functional dependency to the $Nurse_s$ variables. For each nurse n and day d , we have a decision variable $DayWL_{n,d}$ with domain $\{0, \dots, MaxDayWL\}$. This is the number of minutes worked by nurse n on day d .

Other variables represent service time and traveling time: $T_{n,d}^{svc}$ and $T_{n,d}^{trv}$ are, respectively, the service time and the travel time of nurse n in day d ; they also range from 0 to the maximum day workload $MaxDayWL$.

We also have variables containing the weekly totals: each nurse n has a weekly travel time WT_n^{trv} , a weekly service time WT_n^{svc} , and a weekly workload WL_n , that is the sum of the travel and service time; their domain is $\{0, \dots, MaxWeekWL\}$.

Another decision to be taken is the order in which a nurse visits the assigned patients. This problem will be dealt with in Section 2.3.

2.2.2. Constraints

The workload of a nurse n in a day d is the sum of the total service time and the travel time

$$DayWL_{n,d} = T_{n,d}^{svc} + T_{n,d}^{trv}, \quad (1)$$

while the (whole week) workload WL_n is simply the sum of the day workloads:

$$WL_n = \sum_{d=1}^{N_d} DayWL_{n,d} \quad (\forall n \in \mathcal{S}_{nurse}).$$

The service time is the sum of the durations of the services provided by nurse n in day d . In order to compute the service time of each nurse in each day (and to enforce at the same time that the total service time does not exceed the maximum day workload $MaxDayWL$), there can be different constraint models. An intuitive model could be to define a set of Boolean decision variables $X_{n,s}$ (variables taking values *true* or *false*) saying if nurse n takes service s , and add corresponding sum constraints. A more promising model can be obtained noticing similarities with classic problems such as bin packing and (multi)-knapsack problems [17,35].

In a multi-knapsack problem, a set of N_{items} items with size $size_i$ have to be placed in a set N_k of containers without exceeding their capacities.

The global constraint *multiknapsack* (also called *binpacking*) [54] is suited to solve such problems. In

Constraint Programming a multiknapsack problem can be simply defined with the constraint

$$multiknapsack(a, size, load),$$

where

- $a \equiv \{a_i\}$ is a set of decision variables a_i with domain

$$a_i :: 1, \dots, N_k \quad (\forall i \in 1, \dots, N_{items}),$$

with the intended meaning that $a_i = j$ iff item i is assigned to knapsack j ;

- $size \equiv \{size_i\}$ is the array containing the size of each item;
- $load \equiv \{load_j\}$ is a set of constrained variables that represent the actual load of each knapsack, i.e. the sum of the sizes of the items assigned to that knapsack. In order to solve a multi-knapsack problem, the $load_j$ domains are constrained to take values up to the capacity of the knapsacks:

$$load_j \leq capacity_j \\ (\forall j \in 1, \dots, N_{knapsacks}).$$

By using the techniques currently embedded in the *multiknapsack* constraint, Shaw [54] obtains a speedup of several orders of magnitude with respect to the intuitive model, so we decided to take this second approach.

In the HHC problem, we can consider:

- each service s as an item of size dur_s which has to be placed in one of the bins/knapsacks;
- to each pair $(nurse, day)$ we associate a knapsack of capacity $MaxDayWL$;
- the actual load of knapsack $(nurse, day)$ is the service time $T_{nurse,day}^{svc}$ of nurse $nurse$ in day day .

We can thus bind the *Nurse* decision variables with the whole T^{svc} vector. We impose the constraint

$$multiknapsack(Nurse, dur, T^{svc}),$$

which enforces that $(\forall n \in \mathcal{S}_{nurse}, \forall d \in \{1, \dots, N_d\})$:

$$T_{n,d}^{svc} = \sum_{\substack{s \in \mathcal{S}_{serv} \\ d = day_s}} [Nurse_s = n] \times dur_s,$$

where the square brackets are the Iverson brackets [33].¹

The travel time $T_{n,d}^{trv}$ of nurse n in day d is computed by a specific constraint *traveltime*, explained in detail in Section 2.3.

2.2.3. The objective

Ideally, in the HHC problem in Ferrara, one should find a solution that achieves the objectives described in the introduction. As it is often the case in real world applications, the objectives are stated by the users in a blurred, informal way, not clear enough to convert them into a unique objective function, and all functions one can think of are debatable. The main objectives, as stated by the user, are to (i) avoid that a patient is visited by too many different nurses (loyalty) and (ii) balance the weekly workloads of the nurses (but they also have to keep reasonable conditions for all nurses and for the NHS).

We first address the two issues separately, then we combine them.

Loyalty. One way to obtain maximum loyalty is to minimize the number of nurses that visit each patient. For each patient $p \in \mathcal{S}_{patient}$ we link the array of decision variables $Nurse_p$ with the number of different values in it.² The *alldifferent* [48,59] global constraint enforces that all the variables in an array are given different values. The global constraint *NValue* [6,27,60] can be thought of as a soft version of *alldifferent*, and it counts the number of different values in a vector of variables. Various efficient propagation algorithms have been proposed for the *NValue* constraint; using the global constraint *NValue* is usually considered more efficient than using a set of smaller constraints [7]. By counting the different values in the set of decision variables associated to a certain patient p , we obtain the number PN_p of different nurses who take care of that patient:

$$NValue(Nurse_p, PN_p) \quad (\forall p \in \mathcal{S}_{patient})$$

and we can compute the overall loyalty penalty LP summing up the single contributions:

$$LP = \sum_{p \in \mathcal{S}_{patient}} PN_p. \quad (2)$$

¹Let P be a statement, $[P]$ evaluates to 1 if P is true and to 0 otherwise.

²To simplify the notation, we indicate with $Nurse_p$ the subset of the *Nurse* array corresponding to patient p .

Balancing workloads. There are various ways to achieve balanced workloads [55]. A widely used method is the minimization of the maximum workload, so a first proposal is to minimize:

$$BalanceWL' = MaxWL = \max_{n \in \mathcal{S}_{nurse}} (WL_n). \quad (3)$$

There are known pitfalls in adopting the minimization of the maximum (see, e.g., [40]). One simple observation is that two solutions are considered equivalent if they have the same value for the highest workload, regardless of the working conditions of all other nurses.

Consider for instance the following example, also depicted in Fig. 3.

Example 1. Suppose that there is a patient X that lives rather far from the hospital, and also requires a long service time (Fig. 3); suppose that, in order to visit and service him, a nurse takes a time exactly equal to the maximum day workload $MaxDayWL$. The other patients, instead, live close to the hospital and have shorter service times. In such a case, one nurse will serve only patient X , while the other patients will be subdivided among the other nurses.

In this example, the minimization of the maximum workload provides the same value for all the possible assignments: the maximum workload is fixed to the maximum day workload, and corresponds to the nurse visiting patient X .

More recent proposals to obtain balanced solutions are the *spread* [45] and *deviation* [50] constraints. These constraints link a set of decision variables with their variance or with the mean absolute deviation. In this way, it is both simple to define objective functions that minimize the variance (resp. mean absolute deviation), and efficient, thanks to the powerful global constraint propagators. In our application, we decided to experiment with the *deviation* constraint, thus minimizing the L_1 norm although, as noticed in [51], “These criteria are not equivalent: Minimizing



Fig. 3. Example situation: the hospital is depicted with a black square, while patients are depicted as circles. The area of each circle represents the service time required by the patient.

L_1 or L_2 does not lead to the same solutions and it is not always obvious which one to choose. In fact, this is an old and recurrent debate”.

These constraints are very useful when the total workload is fixed, and the objective is only to distribute it fairly to a set of resources. Indeed, in some CP languages the sum of the variables occurring in the *spread* or *deviation* constraint is *required* to be a constant. In our case, however, the workload of the nurses depends also on the travel time, that depends on the partitioning of the patients to the nurses. This can lead to situations in which minimizing the absolute deviation of the workload provides low quality solutions, as shown in the following example.

Example 2. We have 5 patients, named $A, \dots, E$, on the Cartesian plane, as depicted in Fig. 4. The time (in minutes) required to move from a patient to another is simply given by the Euclidean distance. Patients A, B , and C have a service time of 10 minutes, while the other two have a service time of 1 minute. The hospital H is in $(0, 0)$, and there are two available nurses.

If we minimize the maximum workload, we obtain the assignment:

Nurse	Patients	Travel	+ Service = Workload
1	A, B	$\sqrt{26} + 4 + 5\sqrt{2} +$	$20 \simeq 36.17$
2	C, D, E	$20 +$	$12 = 32$

In this case, the absolute deviation is 4.17.

However, consider this assignment:

Nurse	Patients	Travel	+ Service = Workload
1	B, C	$5\sqrt{2} + 10 +$	$20 \simeq 37.07$
2	A, D, E	$\sqrt{26} + \sqrt{116} + 10 +$	$12 \simeq 37.87$

The absolute deviation is only 0.8, so if we minimize the workload deviation, we would definitely prefer this second assignment. However, the second assignment has worse working conditions for *both* nurses: both have a higher workload. This solution is also worse from the National Health Service viewpoint, as it is

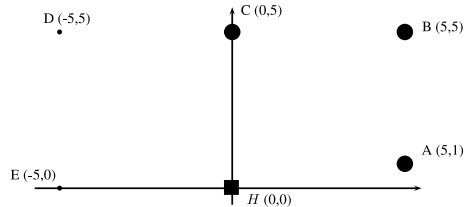


Fig. 4. Example situation: the hospital is depicted with a black square, while patients are depicted as circles. The area of each circle represents the service time required by the patient.

more expensive, since nurses are paid for the whole workload, and they cannot be used for more useful tasks at the hospital.

Finally, the nurses also employ more time driving, producing pollution and traffic.

But there can be even worse situations.

Example 3 (Example 1 continued). In the example in Fig. 3, the nurse servicing patient X has the highest possible workload, and this cannot be reduced. So minimizing the absolute deviation amounts to maximize the workload for the other nurses! Since the total service time is fixed, the only way to increase the workload for the nurses is to increase their travel time, by providing them *the worst possible assignment* (the assignment that maximizes the total travel time).

This is even worse than minimizing the maximum: in this example, the minimization of the maximum considers all solutions as the same; but the deviation of the workload considers as best solutions those in which each nurse is forced to travel uselessly in the whole city area, spending as much time as possible driving from a patient to another. In order to avoid this unreasonable behaviour, one possibility is to remove the travel time from balancing.

Our implementation is based on the Comet constraint programming system [58]. In Comet, the sum of the variables occurring in *deviation* is required to be a constant. For this reason, our objective function could not contain a term $deviation(WL)$, because the sum of the week workloads of the nurses is not fixed (it depends on the nurses' travel time, which, on its turn, depends on the actual partitioning of the patients to the nurses). We decided to remove the travel time from balancing; the absolute deviation is computed only on the service time, and we add a term containing the total travel time:

$$BalanceWL'' = deviation(WT_n^{svc}) + \sum_{n,d} T_{n,d}^{trv}, \quad (4)$$

where WT_n^{svc} is the week service time of nurse n , computed simply as

$$WT_n^{svc} = \sum_{d=1}^{N_d} T_{n,d}^{svc}$$

and

$$deviation_n(WT_n^{svc}) = \sum_{n=1}^{N_n} \left| WT_n^{svc} - \sum_{k=1}^{N_n} \frac{WT_k^{svc}}{N_n} \right|.$$

This solves the problem highlighted in Examples 2 and 3: since the total travel time appears outside the *deviation*, the solver will try to minimize it. On the other hand, minimizing Eq. (4) does not ensure that a minimum of the deviation of the workload is found, because, in general

$$\begin{aligned} & deviation_n(WT_n^{svc}) + \sum_{n,d} T_{n,d}^{trv} \\ & \neq deviation_n \left(WT_n^{svc} + \sum_{n,d} T_{n,d}^{trv} \right). \end{aligned}$$

Combination of the objective functions. As we explained, we have two criteria to be minimized: one tries to provide solutions with a good *loyalty*, while the other tries to balance the workload of the nurses, without increasing uselessly the travel time of the nurses.

There are many ways in the literature to combine two criteria; we adopted a widely used method, namely a weighted sum of the two components:

$$minimize(\alpha_1 \cdot BalanceWL + \alpha_2 \cdot LP). \quad (5)$$

α_1 and α_2 are positive real numbers that can be chosen by the user in order to reflect the current priorities adopted in the AUSL. Of course, such values can be tuned later on in order to better suit the decision maker preferences. *LP* is the Loyalty Penalty (see Eq. (2)), while the term *BalanceWL* balances the workload of the nurses and will be instantiated either to *BalanceWL'* or to *BalanceWL''*, as will be clear soon.

We proposed two ways to balance the workload; so we will have two objective functions to experiment with.

The first is the minimization of the maximum workload (Eq. (3)), so the objective function in Eq. (5) becomes:

$$\begin{aligned} OF_1 &= \alpha_1 \cdot \max_{n \in \mathcal{S}_{nurse}} (WL_n) + \alpha_2 \cdot LP \\ &= \alpha_1 \cdot \max_{n \in \mathcal{S}_{nurse}} \left(\sum_{d=1}^{N_d} T_{n,d}^{svc} + T_{n,d}^{trv} \right) \\ &\quad + \alpha_2 \cdot LP. \end{aligned} \quad (6)$$

The second method is in Eq. (4), so the objective function OF_2 becomes:

$$\begin{aligned} OF_2 &= \alpha_1 \left(deviation_{n \in \mathcal{S}_{nurse}}(WT_n^{svc}) + \sum_{n,d} T_{n,d}^{trv} \right) \\ &\quad + \alpha_2 LP \\ &= \alpha_1 \left(deviation_{n \in \mathcal{S}_{nurse}} \left(\sum_{d=1}^{N_d} T_{n,d}^{svc} \right) + \sum_{n,d} T_{n,d}^{trv} \right) \\ &\quad + \alpha_2 LP. \end{aligned} \quad (7)$$

In Section 3, we will compare experimentally which of the two objective functions (OF_1 and OF_2) performs best in our problem.

2.3. Addressing the routing

In order to satisfy the constraint on the maximum day workload, one should compute both the service time and the travel time of a nurse's daily shift (Eq. (1)). On the other hand, computing the travel time of a nurse implies computing the optimal route that connects the hospital and all the patients assigned to that nurse on that day, which is a (hard) optimization problem on its own, known as the Traveling Salesman Problem (TSP).

We embedded a TSP solver inside a (user-defined) constraint *traveltime*, so that it can perform constraint propagation during search of the main problem (i.e., the HHC). Notice that in a given node of the search tree, some of the patients will be assigned to a nurse n , while others will still be unassigned; the constraint *traveltime* should compute the bounds of the travel time, namely the minimal travel time of the nurse (assuming that nurse n will be assigned a minimal number of patients, amongst those in the domain), and the maximum one (assuming that nurse n will be assigned all the possible patients in the domain). In order to compute the travel time $T_{n,d}^{trv}$ of a nurse n in day d , we need to provide to such constraint:

- (1) the patients associated to the services pat_s ,
- (2) the matrix of distances D , providing the travel time between patient locations as well as to and from the hospital,
- (3) and the subset $Nurse|_{day=d}$ of the decision variables (assigning nurses to services) that correspond to the services to be provided in day d , i.e.,

$$\begin{aligned} Nurse|_{day=d} \\ \triangleq \{Nurse_s | s \in \mathcal{S}_{serv}, day_s = d\}. \end{aligned}$$

Note that, since each patient is assigned a unique location, once the patient of a service is given also the location where the service must be delivered is known. As a recap, the actual parameters are:

$$T_{n,d}^{trv} = \text{traveltime}(\text{pat}_s, D, \text{Nurse}|_{\text{day}=d}).$$

Operationally, the *traveltime* constraint awakes every time one of the variables in $\text{Nurse}|_{\text{day}=d}$ is instantiated, i.e., when a nurse is assigned to a service. The constraint computes the TSP corresponding to the patients that must be visited by nurse n in day d . This provides a valid lower bound to the actual travel time, and can be used in a branch-and-bound search. When all the $\text{Nurse}|_{\text{day}=d}$ variables are ground, the cost of the TSP becomes the real travel time, and we are able to fix the value of $T_{n,d}^{trv}$ to the TSP cost. In our implementation, the constraint implements directed bound consistency from the variables in $\text{Nurse}|_{\text{day}=d}$ to the $T_{n,d}^{trv}$ variable, considering only the lower bound (as the upper bound would not perform significant pruning in our case). Even if recomputing a TSP on each instantiation may sound computationally expensive, preliminary computational results showed that this choice provides better results than heuristically inserting the new node, by way of a cheapest insertion procedure, in the tour previously computed for the other nodes. This is probably due to the fact that the TSP instances to be solved at each step are rather small, so that the TSP solver we embedded in the *traveltime* constraint is quite fast. Moreover, we use a caching mechanism to avoid recomputing TSPs that have been solved previously during the exploration of the search tree.

Regarding TSP solvers, one may either implement one, e.g. in Constraint Programming [4,31], or use an off-the-shelf solution. We found that implementing a solver in pure CP slowed down significantly the search (as also witnessed in the literature [10]). The state of the art in the area is by no doubt Concorde [1], which relies on integer linear programming with the addition of specific TSP-targeted features. However, Concorde requires an ILP solver and is oriented to large instances, while the number of patients visited by a nurse in a single day in our case is rather limited, usually far below 15. Furthermore, Concorde is not easily customizable but it can only be used as a black box, so that it would be impossible to extend our solution method in a further development in order to handle additional constraints on the services, such as time windows or precedence constraints. Therefore, we adopted a solver based on Lagrangian Metaheuristics [25] which per-

forms very well on small TSP instances, would allow us to add further customizations in the future if needed, and proved to be suitable for integration in our system. In the following we provide a brief description of the method. A full description can be found in [24].

Lagrangian Relaxation (LR) is a very well-known technique in mathematical programming that computes usually tight bounds to optimization problems by moving hard-to-satisfy constraints into the objective function and penalizing their violation [15]. In the TSP case, Held and Karp [28] were the first to exploit the fact that a Hamiltonian circuit (a circuit that passes once by each node) on a n -nodes graph can be described as a connected subgraph of n edges where each node has exactly two incident edges (degree 2). By relaxing the degree constraints, the remaining problem is easily (polynomially) solvable since it consists of computing a minimum cost one-tree, i.e., a spanning graph with exactly one cycle. LR removes the degree constraints and adds a penalty to the objective function for any vertex whose degree is not 2. Each degree-constraint violation is multiplied by a so called *Lagrangian multiplier*. For each set of multipliers, the optimal solution of the relaxed problem provides a lower bound on the optimal value of the original TSP instance. The multipliers can be tuned to compute the best (highest) of such lower bounds. In the TSP case, the search for the best multipliers reduces to the maximization of a concave polyhedral function for which several methods are available, among which the subgradient algorithm [22]. The TSP solution approach described in [25] combines the subgradient optimization technique, which is used to search for the best multipliers, together with a fixing procedure, that restores the feasibility of the relaxed solution by turning the current one-tree solution into a Hamiltonian circuit. An upper bound as well as a lower bound are thus provided at each iteration of the subgradient algorithm. Computational experiments in [25] show that for reasonable sized instances the procedure quickly converges and closes the gap between lower and upper bound, thus yielding the optimal TSP solution.

2.4. Search strategies

In Constraint Programming, domain filtering, achieved by the imposed constraints, has to be coupled with search tree exploration in order to reach solutions. The search strategy design is responsible for the shape of the tree and the way to explore it, the aim being in-

corporating knowledge of the problem and its structure to rapidly drive the search towards (good) solutions.

Since the decision variables are the nurses associated to the services, we have to select a variable (a service) and assign a nurse to it. Since all nurses can provide equivalent services, the constraint model has a large number of symmetries: given a solution, a permutation of the nurses provides another equivalent solution. We decided to break symmetries during search, with a variation of the technique proposed in [38]. In particular, the first service is always assigned the first nurse; the n th service can be either associated to one of the already employed nurses, or to the first not-yet-employed nurse. In fact, trying all the possible combinations of the not-yet-employed nurses is useless, as they can provide only symmetric solutions. Formally, if $E_{n-1} = \{Nurse_k | k \in 1, \dots, n-1\}$ is the set of nurses employed for the first $n-1$ patients, then

$$Nurse_n \in E_{n-1} \cup \{1 + \max E_{n-1}\}.$$

Given this definition of the search space, we applied several search strategies to our problem; we first defined a general purpose one (i.e., a predefined search strategy, that can be applied to any problem), then we developed one more tailored for the problem at hand.

A first, general purpose search strategy, called in the following **GSRFF** (Generic Search with Restarts and First Fail), performs a depth first search selecting the variable to assign, at each node, with the First Fail heuristic [26], i.e., it selects first the variable with the smallest domain. The value to assign is selected randomly among the ones in the domain. A widely used technique to improve the efficiency of a search strategy is using *restarts* [30], in order to avoid that wrong decisions taken near the root can drive the search into a bad area. This technique was shown to be very effective in presence of a so-called heavy-tail behavior [23,29]. Using restarts means that after some pre-defined timeout the search is stopped and started again, possibly with a different timeout. In the GSRFF search strategy, we apply restarts with the optimal timeout sequence proposed by [37], as also suggested in [56].

We also devised a search heuristic more tailored to the problem structure (**HEUR**). The variable selection criterion selects first variables related to services with bigger durations, which is a sensible choice when dealing with *multiknapsack* problems. Let the selected variable be associated to service s of patient pat_s , the value selection heuristic tries to assign a nurse who is already visiting pat_s in the attempt to keep low the loy-

alty penalty LP (see Eq. (2)). If more than one nurse was already assigned to that patient previously, the one with the lowest week workload WL is selected, in the attempt to keep workloads balanced.

This heuristic can also be adopted in the context of a Large Neighborhood Search (**H + LNS**), a technique that hybridizes CP and Local Search [53]. The idea is to perform local search with a large neighborhood, and to explore the neighborhood with CP.

The Comet programming language allows the programmer to adapt a CP model into a hybridized procedure that scales very well with the problem size, and which explores a neighborhood large enough to avoid the need for a metaheuristic.

Our LNS procedure proceeds as follows.

- (1) First, a feasible solution is found, using CP; in our case we use the HEUR heuristics explained earlier.
- (2) Then, a part of the current solution is relaxed, by:
 - undoing the assignment on a set of variables (i.e., by restoring their domain to the initial setting); we decided to un-assign 10% of the main decision variables, namely the $Nurse_s$ variables that represents the assignment of nurses to services;
 - fixing the remaining variables to their current value.
- (3) The restricted problem is re-optimized using CP with a limit on the number of failures. Again, we used the HEUR heuristics, with 50 failures as a limit.
- (4) If the current best solution was not improved in the last step, then we go back to step 2.
- (5) Steps 2–4 are repeated until we reach the given timeout (10 minutes).

3. Experiments and results

The implementation is based on the Comet [58] Constraint Programming system. All tests were performed on a computer equipped with a 3.4 GHz Intel Core i7 processor and 7 GB of RAM.

3.1. Search strategy comparison

The instances were provided by the AUSL 109 and represent four weeks of February 2010. In the first set of experiments, the aim is to compare the constraint model to the Hand Made Solution (HMS)

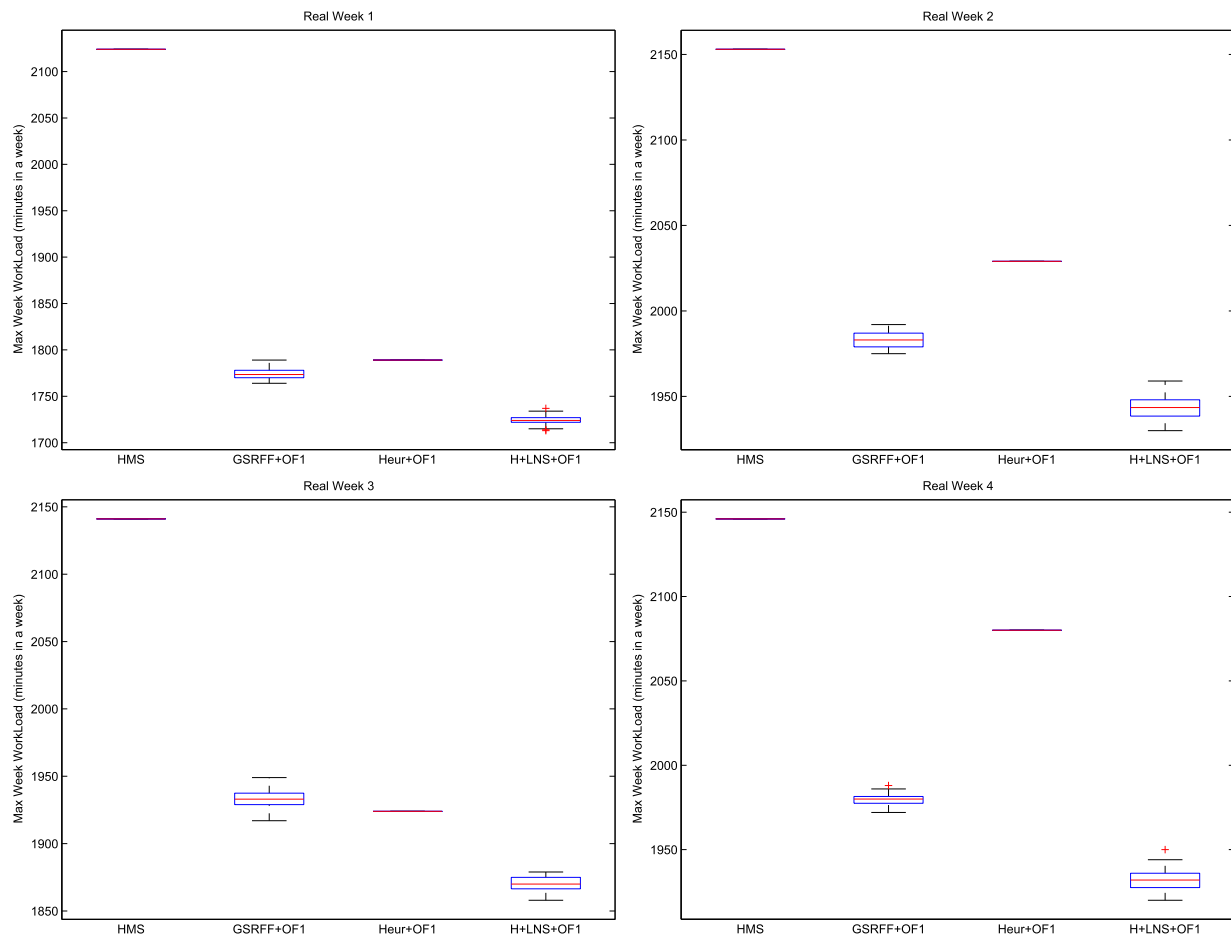


Fig. 5. Maximum Week Workload (the lower the better) of the nurses in the four real instances, computed by the various algorithms, and compared to the Hand-Made Solution (HMS). (Colors are visible in the online version of the article; <http://dx.doi.org/10.3233/AIC-140632>.)

adopted by the nurses in 2010. In these experiments, we used OF_1 as objective function (Eq. (6)), considering $\alpha_1 = \alpha_2 = 1$. We show the results of the various search strategies defined in Section 2.4; all algorithms were run with a 10 minutes timeout. Each of the algorithms was executed 40 times; the results are shown in a set of box-plots.

In Fig. 5, we show the Maximum Week Workload of the nurses obtained by the best solution found by the various algorithms within 10 minutes. The first observation is that all algorithms were able to improve significantly on the Hand-Made Solution in each instance and in each run. The generic search algorithm (GSRFF) is very competitive, but the best algorithm is always the Large Neighbourhood Search combined with our heuristic (H + LNS + OF_1). Algorithm HEUR is deterministic, so each run provides exactly the same result.

Figure 6 shows the Loyalty Penalty (Eq. (2)) obtained by the same algorithms in the four real instances. Considering the loyalty, the GSRFF search algorithm was not able to improve on the Hand-Made Solution. This is probably due to the fact that the Hand-Made Solution is based on the a-priori partitioning of the patients according to zones, as explained in Section 1.2, so a patient gets always the same nurse, unless the workload of that nurse exceeds the working hours (in such a case, some patient gets reassigned to other nurses). On the other hand, since the instances have very tight schedules, and the actual workload gets very close to the maximum working hours, some re-assignment takes place, and the greedy procedure followed by the nurses is not able to obtain the best possible solution. The heuristic we developed for this problem (HEUR), in fact, gets always better loyalty scores than the Hand-Made Solution, and using Large

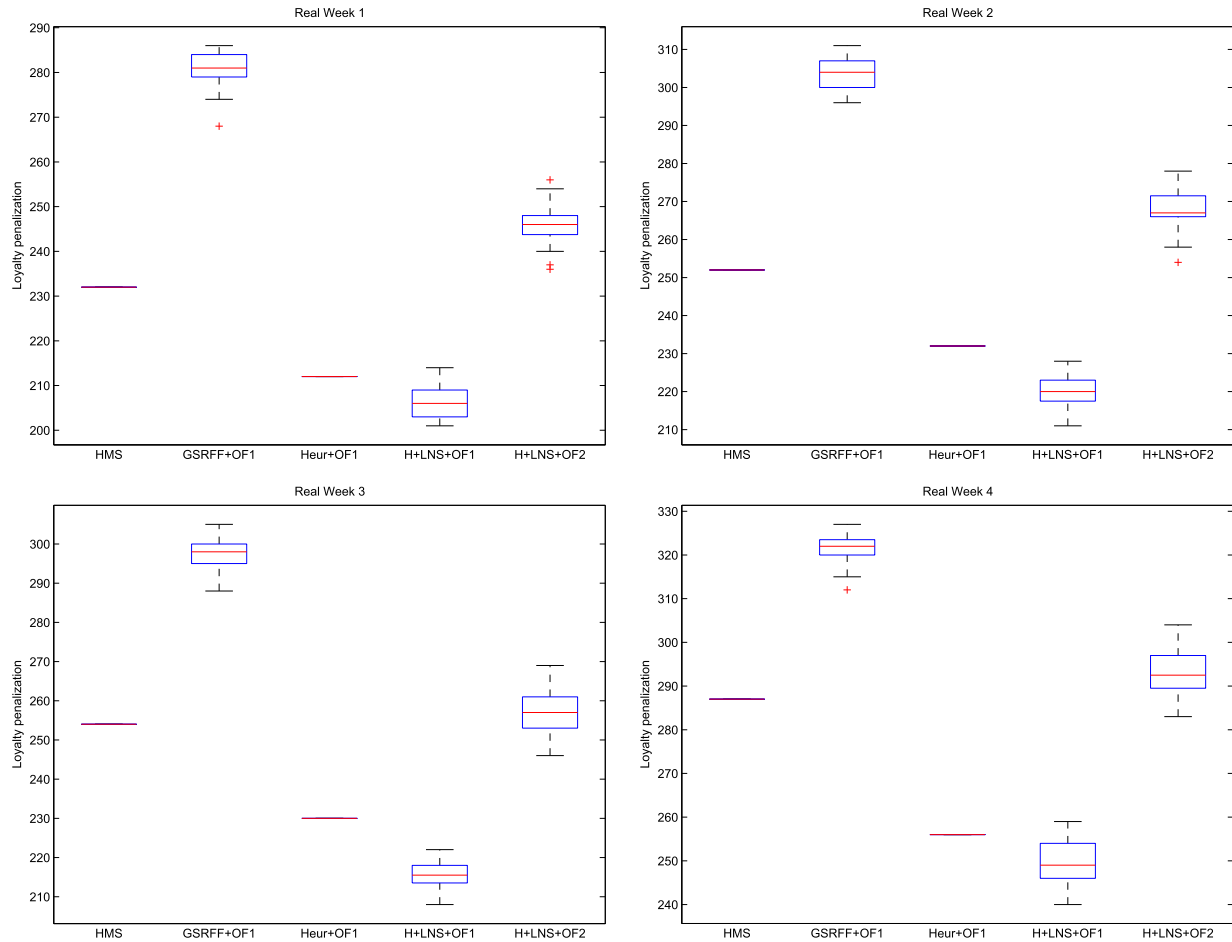


Fig. 6. Loyalty penalization (the lower the better) obtained by the algorithms in the four real instances, compared with the Hand-Made Solution (HMS). (Colors are visible in the online version of the article; <http://dx.doi.org/10.3233/AIC-140632>.)

Neighbourhood Search provides a further improvement.

3.2. Sensitivity analysis

After studying the efficiency of the various search algorithms, we did a trial concerning the relative importance of the two components of the objective function OF_1 . As shown in Eq. (6), the objective function OF_1 is the weighted sum of two components. We asked the users about reasonable values for the coefficients α_1 and α_2 , but they were not able to provide values nor motivations for them, although they desired that both components were taken into consideration. When the preferences of the decision maker are totally unknown, a rigorous approach would be to compute the whole Pareto frontier [19], that is the set of all the non dominated solutions in the space of the two criteria. How-

ever, computing the Pareto frontier is much more demanding from a computational viewpoint, and a common approach is that of a weighted sum. We chose, as tentative values for the weights, $\alpha_1 = \alpha_2 = 1$; however in the developed application these parameters can be varied by the user. We are now interested in performing some kind of sensitivity analysis to examine where our solution would be placed with respect to the Pareto front, in order to get some hints on the impact of each of the criteria and how the solution would change if slightly altering the weights coefficients. Of course, a rigorous analysis is out of reach, since the Pareto front can only be approximated when working with incomplete search methods or when using timeouts (as we do, and it is standard practice in HHC related problems, see Section 4).

We approximate the Pareto frontier with a classical method [61]. The procedure exploits the fact that

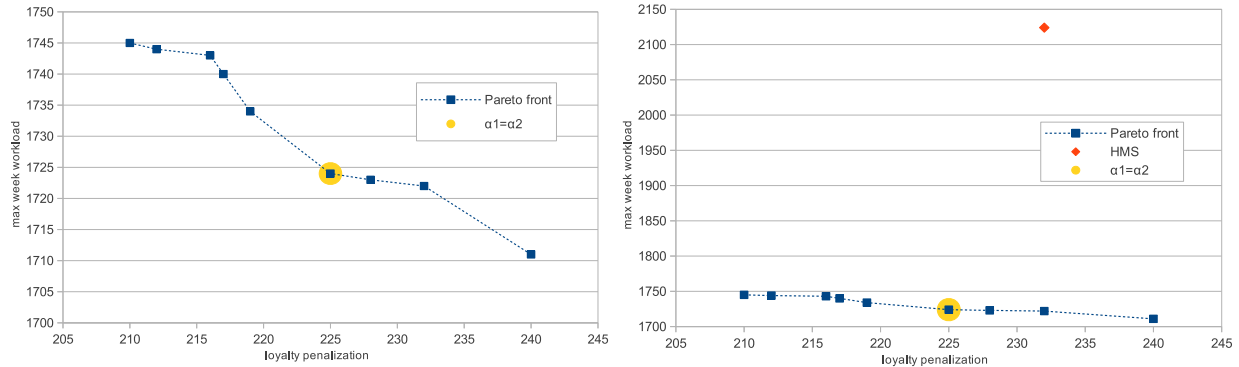


Fig. 7. (Near) Pareto front of solutions of one weekly instance, reporting also the hand-made solution (HMS). The highlighted point is the one selected if $\alpha_1 = \alpha_2$. (Colors are visible in the online version of the article; <http://dx.doi.org/10.3233/AIC-140632>.)

the domain of (at least) one of the two criteria is integer. In a minimization problem, the idea is to solve a sequence of single objective optimization problems where one criterion is optimized and the other is constrained to be less than the value of that criterion obtained at the previous solution. In our case, we kept the maximum weekly workload as the objective to be minimized and constrained the value of the loyalty component LP . As both have integer domains, the reverse would be a feasible choice too. In practice, at the first iteration the problem is solved by setting $\alpha_1 = 1$ and $\alpha_2 = 0$ in Eq. (5). This yields the solution at the extreme right-hand side in Fig. 7. Let $LP^{*(1)}$ denote the value of the loyalty component associated to the solution found at iteration 1. At iteration $k = 2$, the same problem is solved with the addition of the constraint $LP < LP^{*(k-1)}$, so that the solution with lowest maximum weekly workload among those with lower loyalty is found. This yields the next point going leftwards along the Pareto front in Fig. 7. Of course, in case we find a point that dominates a previously found one, the dominated is removed from the Pareto set. The procedure iterates until no feasible solution is found.

On the left side of Fig. 7 we present the approximation of the Pareto front obtained by the above mentioned procedure using search H + LNS (Section 2.4) on one weekly instance with a 10 minutes timeout (Intel Atom Processor N450 1.66 GHz and 1 GB of RAM). The solution computed by setting $\alpha_1 = \alpha_2 = 1$ is highlighted. One can see that, at least for this instance, such solution is a good compromise, about in the middle of the Pareto graph; in this solution a small reduction in the maximum workload should be traded by a consistent deterioration of loyalty. The width of the Pareto graph is 30 units (it

ranges from 210 to 240 in the loyalty penalization) and its height is 34 units (from 1711 to 1745 in the maximum week workload), so the choice of coefficients $\alpha_1 = \alpha_2 = 1$ does not seem unreasonable (although, of course, the graph could be different in other instances).

On the right of Fig. 7, the (near) Pareto front is scaled to depict the hand-made solution (HMS) for the same instance; it can be noted that it is quite far from the (near) Pareto front, and that it is dominated by many points in that set.

While we are well aware of the limitations of using a scalar function to address a multi objective problem, we believe that, for our instances, the proposed method is able to reach good compromises between the two criteria.

3.3. Objective function comparison

We now evaluate the effect of using the objective function OF_2 , defined in Eq. (7), instead of OF_1 , given in Eq. (6). The objective stated by the user was a fair balancing of the workload between the nurses; this was implemented in OF_1 by adopting the minimization of the maximum workload as a balance criterion, while in OF_2 we use the absolute deviation from the mean of the service time, plus the total travel time. In Fig. 8 we show the absolute deviation from the mean of the week workload of the various nurses. As expected, the hand-made solution is far from the optimum, and results the most unfair of the considered strategies. The algorithm adopting OF_1 gives lower deviation of Week Workload than OF_2 . This is probably due to the fact that in OF_2 we minimize the deviation of the Service Time, so the Travel Time is not included in the balancing.

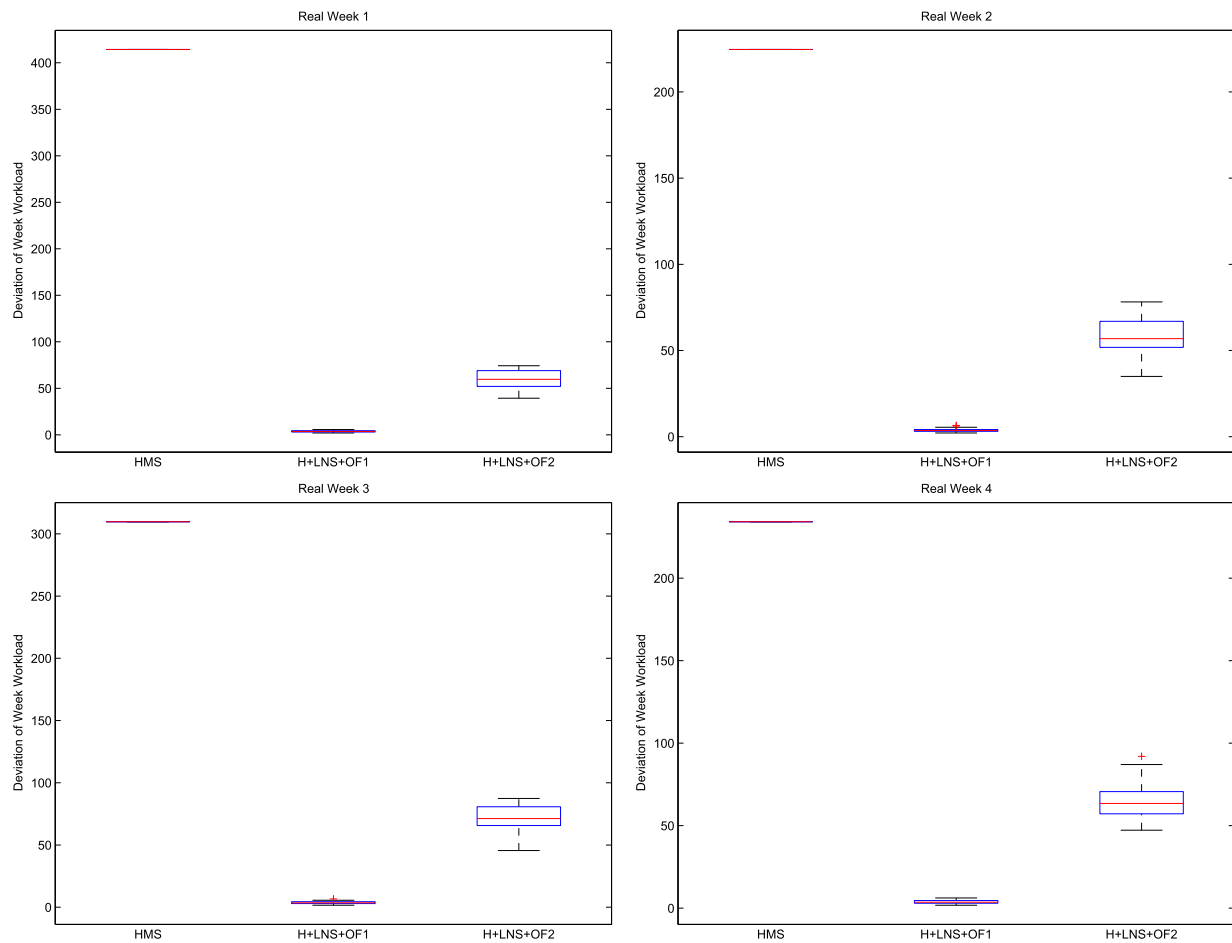


Fig. 8. Absolute deviation from the mean of the nurses' week workload (the smaller the better) obtained with the two objective functions in the four real instances, compared with the Hand-Made Solution (HMS). (Colors are visible in the online version of the article; <http://dx.doi.org/10.3233/AIC-140632>.)

In order to investigate further how the objective function impacts on the distribution of the workload, in Fig. 9 we show a comparison of the best found solutions in a typical week. The box-plot shows the distribution of the week workloads to the various nurses: each data item used in the box-plot represents the week workload of one of the nurses, obtained in the best solution found by each of the two algorithms. As we can see, using as balancing criterion the maximum week workload (as in Eq. (3)) provides a very narrow distribution of the week workload (column $H + LNS + OF1$ in Fig. 9), while minimizing the absolute deviation from the mean of the service time (as in Eq. (4)) provides an unfair distribution of the workload (column $H + LNS + OF2$ in Fig. 9). Again, this is due to the fact that one of the components of the workload, namely the travel time, could not be included in the *deviation*.

Another comparison between the objective functions can be seen by comparing Figs 10 and 11. Figure 10 provides the absolute deviation of the Service Time (ST) in a typical instance, as provided by the proposed algorithms. We can see that through the *deviation* constraint we were able to provide the best values of absolute deviation of the service time (see column $H + LNS + OF2$ in Fig. 10). Unluckily, the same algorithm also provides the worst balancing of the week workload (Fig. 11), that was the real objective of the users.

Considering the loyalty criterion, from Fig. 6 we can see that $H + LNS + OF1$ provides better values than $H + LNS + OF2$.

Finally, we consider the total travel time in Fig. 12. The Hand-Made Solution provides very low values of travel time, as the nurses used as criterion a partitioning of the patients into contiguous zones. Also, OF_2 pro-

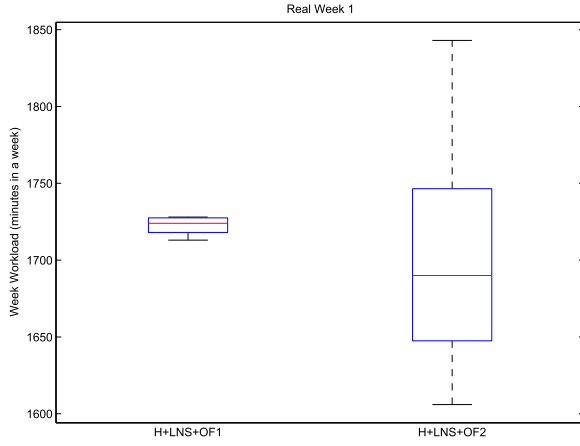


Fig. 9. Distribution of the week workload to the various nurses in the best solutions found using the two objective functions. (Colors are visible in the online version of the article; <http://dx.doi.org/10.3233/AIC-140632>.)

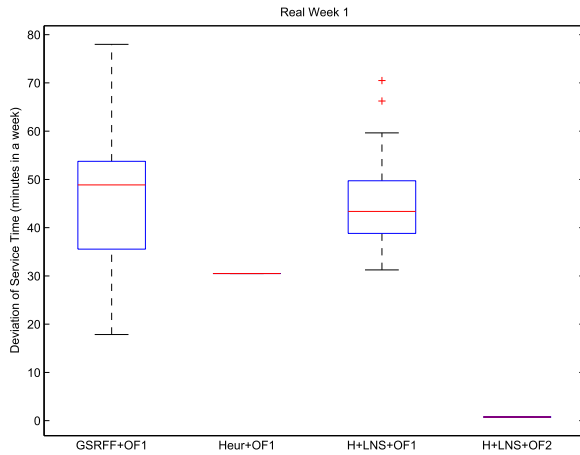


Fig. 10. Absolute deviation from the mean of the service time obtained by the various algorithms in real week 1. (Colors are visible in the online version of the article; <http://dx.doi.org/10.3233/AIC-140632>.)

vides better values than OF_1 , probably because only the travel time of the nurse with the highest workload is important in OF_1 , while OF_2 tries to improve the travel time of all nurses.

3.4. TSP solver

Finally, we did some experiments to assess the efficiency of the adopted TSP solver with respect to other techniques. We considered the hand-made solution in all the four instances. Each solution of the HHC contains one TSP for each nurse for each day, for a total of 240 TSP instances ($4 \text{ weeks} \times 12 \text{ nurses} \times 5 \text{ days}$).

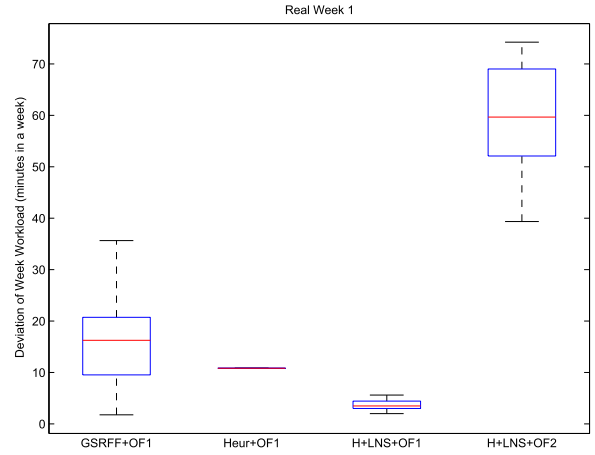


Fig. 11. Absolute deviation from the mean of the week workload obtained by the various algorithms in real week 1. (Colors are visible in the online version of the article; <http://dx.doi.org/10.3233/AIC-140632>.)

We solved all these TSPs with the Lagrangian Relaxation technique described in Section 2.3, with a program written for the Answer Set Programming (ASP) solver *clingo* [20], and with a CP approach written in Comet. The total time needed to solve all the instances is the following:

- Lagrangian Relaxation: 0.485 s,
- ASP: 24405.22 s,
- Comet: 1.12 s.

Lagrangian Relaxation was the fastest amongst the considered technologies.

We also computed the percentage of time required by TSP solver with respect to the rest of the solution process. We obtained that, in a typical run of the HHC solution process, the LR algorithm took about 66 s, out of a total running time of 698 s (including CP and LR), so LR took about 9.46% of the running time. This shows that even if we changed the TSP solver, moving to a faster one (e.g., a commercial one), we could not reduce the running time by more than 10%.

3.5. Crafted instances

Although the best evaluation of an algorithm is based on real instances, in order to perform a wide and systematic evaluation of our algorithms, we generated a set of crafted instances by randomly selecting patients from the real instances. In particular, we wanted to evaluate the efficacy and efficiency of the algorithms varying two possible indicators of hardness of an in-

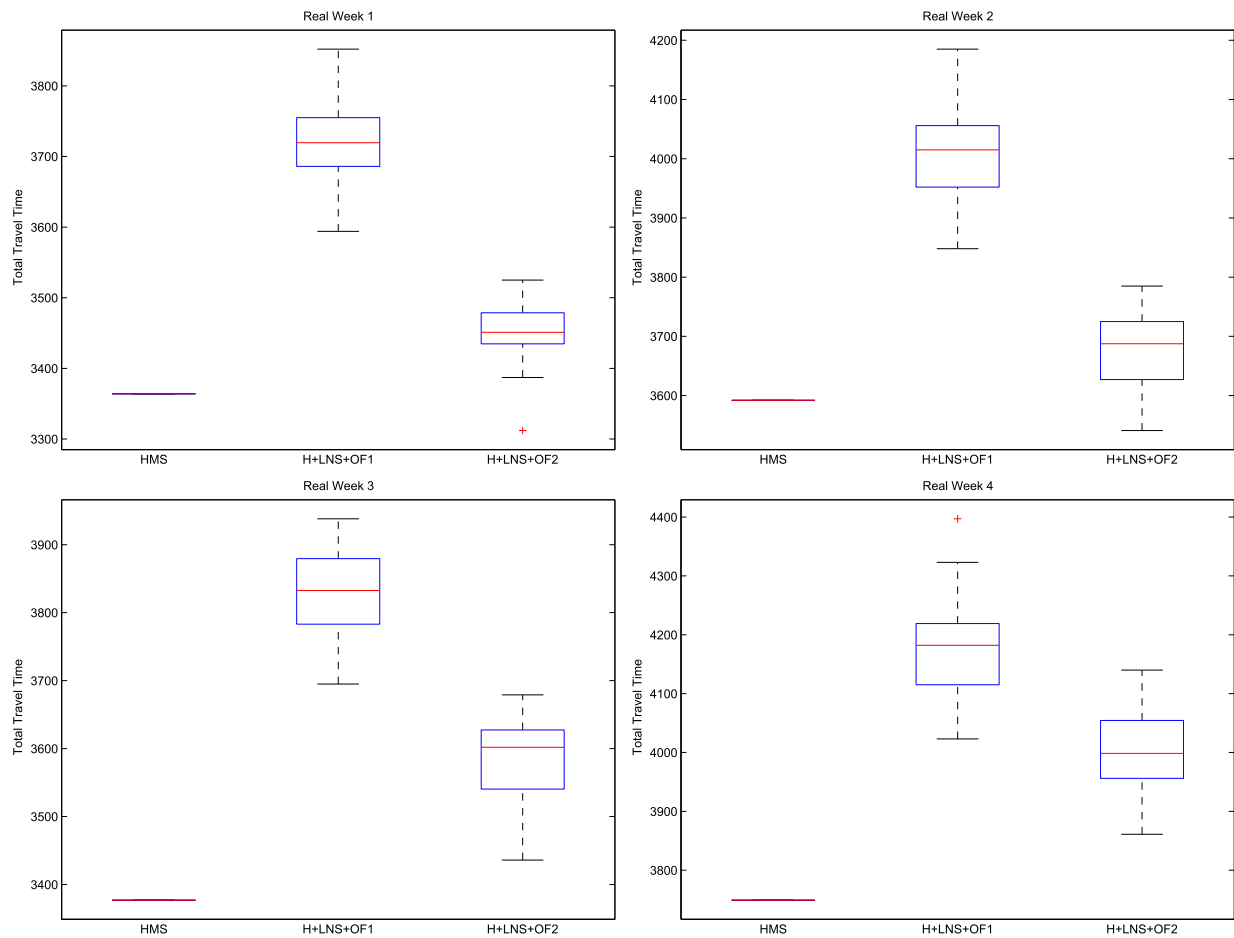


Fig. 12. Total Travel time of the nurses (the smaller the better) obtained with the two objective functions in the four real instances, compared with the Hand-Made Solution (HMS). (Colors are visible in the online version of the article; <http://dx.doi.org/10.3233/AIC-140632>.)

stance: the total number of patients per day, and the daily total service time requested by the patients.

Since the real instances have an average of 65 patients per day with a maximum and a minimum of 82 and 51 respectively, we generated instances varying the number of patients from 50 to 80. To generate an instance, we selected at random the given number of patients; then the instances were classified into groups according to the obtained total service time. In this way, given the services on the real instances we were able to obtain 12 classes of instances, shown in Table 2; for each class we considered 20 instances.

In the following, we report the behaviour of the various algorithms we developed, and, for the sake of comparison, we implemented an algorithm that mimics the greedy algorithm used by the nurses in the Hand Made Solution (HMS).

Note that this generation strategy does not ensure that each instance has a solution. However, it is in-

teresting to note which algorithms were able to solve most instances, when the instance become tighter. Table 3 shows the number of instances that were solved for each algorithm:

Given algorithms $H + LNS + OF1$, $H + LNS + OF2$ and HEUR solved the same number of instances, we report only one column: HEUR. The algorithms based on LNS are good to improve a solution, based on local search from an initial solution. When the instance becomes tighter, the heuristic does not find any initial solution, so the three algorithms were not able to find any solution. The algorithms able to find most solutions are the greedy one, that mimics the hand made solution, and the GSRFF, probably because it includes restarts, which help finding feasible solutions. However, the real instances are not that tight, so for the practical application the algorithms based on LNS are preferred. On the other hand, the HMS algorithm fails to find a solution in one small-sized instance.

Table 2
Classes of instances in the crafted set

Patient No.	Total Service Time							
50	2700–2900	2900–3100						
60		2900–3100	3100–3300	3300–3500	3500–3700			
70				3300–3500	3500–3700	3700–3900	3900–4100	4100–4300
80							3900–4100	

Table 3
Number of solved instances in the crafted set

Patient No.	Svc. Time	GSRFF	HEUR	HMS
50	2700–2900	20	20	20
50	2900–3100	20	20	19
60	2900–3100	20	20	20
60	3100–3300	20	20	20
60	3300–3500	20	20	20
60	3500–3700	20	20	20
70	3300–3500	20	20	20
70	3500–3700	20	20	20
70	3700–3900	20	20	20
70	3900–4100	20	20	20
70	4100–4300	17	8	19
80	3900–4100	20	11	20

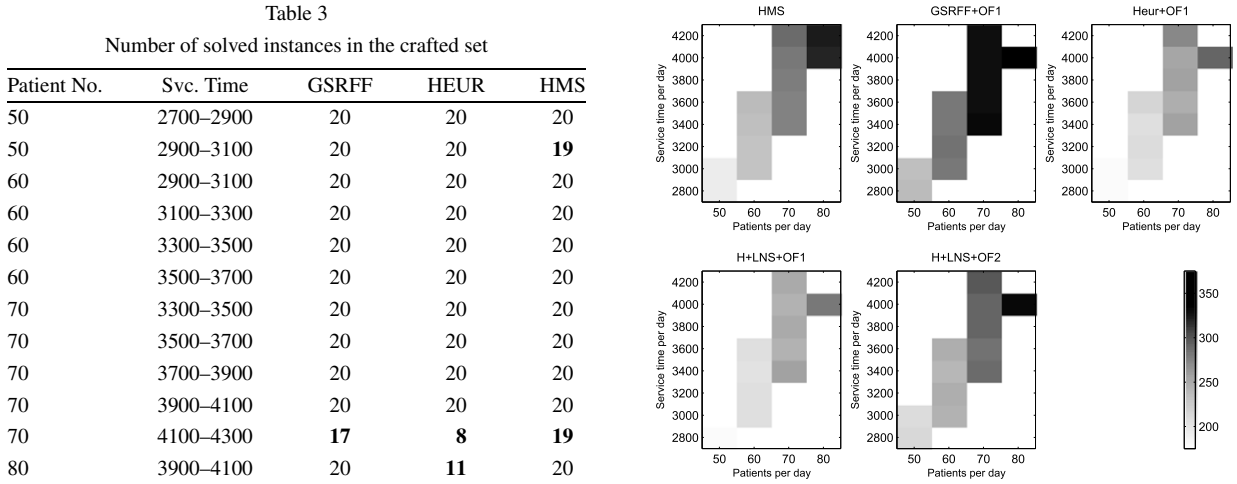


Fig. 14. Average of Loyalty Penalty in the 12 classes of instances.

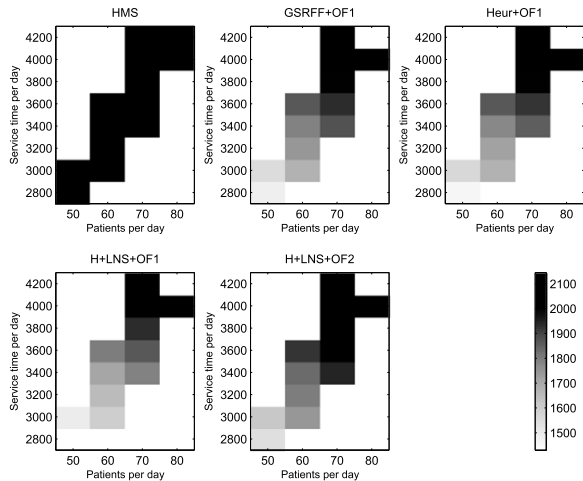


Fig. 13. Average of Max Week Workload in the 12 classes of instances.

Figures 13–15 represent colour maps of the 12 types of instances, where the x -axis is the number of patients per day and the y -axis is the total service time per day. The map is colored according to the average value of 10 runs for 3 randomly selected instances of each class.

As can be seen in Fig. 13, all algorithms are able to improve the hand-made solution regarding the minimization of the Maximum Workload. The generic

search strategy (GSRFF) is already able to improve on the hand made solution, using a tailored heuristics (HEUR) gives a further improvement, and Large Neighbourhood Search (H + LNS) improves further. Not surprisingly, using as balancing criterion the deviation of the service time (Eq. (4), map H + LNS + OF2 in Fig. 13) does not give low values for the maximum workload. Of course, the workload increases as the requested service time increases, but also the number of patients has a strong effect, probably due to the increased travel time.

Considering the *loyalty* (Fig. 14), we have that the GSRFF search algorithm achieves values worse than the Hand-Made Solution (although it is able to improve the sum of the two terms of Eq. (6) with respect to the HMS). In the Hand-Made Solution, patients are partitioned according to zones, so the HMS strategy is directed towards achieving good loyalty, since a patient is likely to have always the same nurse. However, if the workload of some nurse exceeds the maximum working hours, some patients are redistributed amongst the other nurses, so the loyalty term is not necessarily optimal and there could be still room for improvement. The other algorithms, in fact, are able to improve also the *loyalty* term of Eq. (6). Again, the tailored search

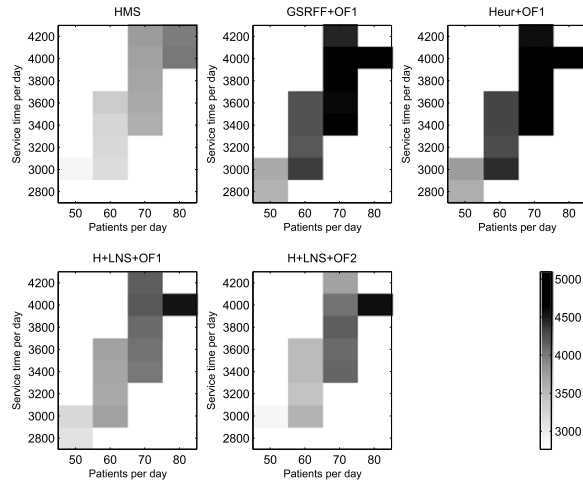


Fig. 15. Average of Total Travel Time in different types of instances.

strategies improve significantly the loyalty. In Fig. 14 we can also see that the loyalty depends mainly on the number of patients, and does not depend significantly on the total service time.

For the *Total Travel Time* (Fig. 15), we can see that the hand-made solution is able to obtain very good solutions. This is due to the fact that patients are partitioned into contiguous zones, and, in the algorithm that mimics the nurses' partitioning, we assumed that each nurse computes the optimal TSP (which is not always the case in real situations). The best algorithm is $H + LNS + OF2$, due to the fact that the sum of the nurses travel times appears explicitly as a term of the summation in the objective function (see Eq. (4)), while the other algorithms strive to optimize the maximum workload, possibly overlooking the workload of the less-occupied nurses. As the intuition suggests, also the travel time depends strongly on the number of patients and weakly on the total service time (Fig. 15).

Finally, we compare the absolute deviation from the mean of the weekly workloads of the nurses. In Fig. 16, we report the box-plots of the results from all the instances, including both the four real instances and the 12×20 crafted ones, for a total of 244 instances. The hand-made solution is not reported because it gave values about 80 times higher, that would make the graph unreadable. There is a clear pattern showing that algorithm $H + LNS + OF1$ performed best. We also plot in Fig. 17 the best values of absolute deviation of the service time; as expected we can see that algorithm $H + LNS + OF2$ obtains the best results. Varying the class of the instance (in the 12 classes), did not show any significant pattern, so the color maps are not reported.

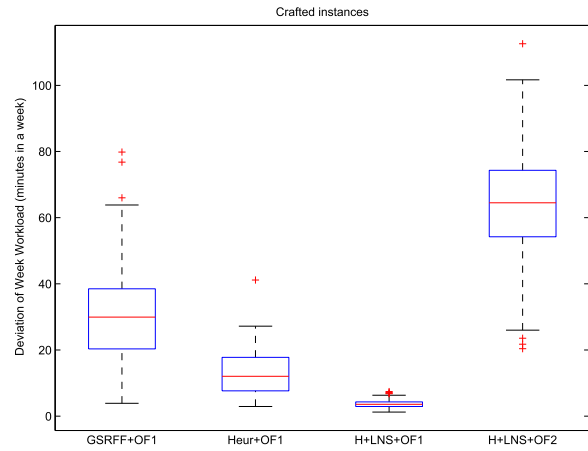


Fig. 16. Absolute Deviation from the mean of the Week Workload of the nurses. Value computed considering all the instances. (Colors are visible in the online version of the article; <http://dx.doi.org/10.3233/AIC-140632>.)

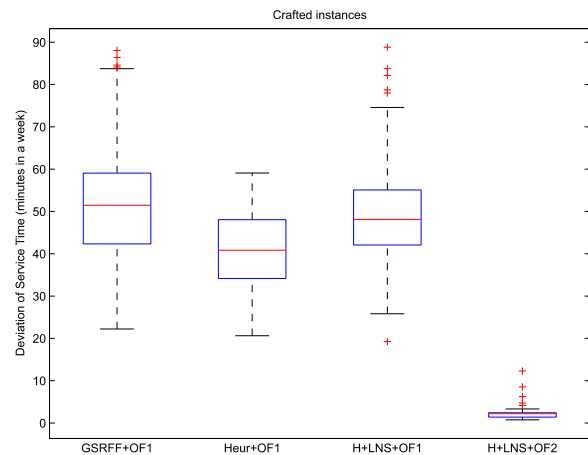


Fig. 17. Absolute Deviation from the mean of the Service Time of the nurses. Value computed considering all the instances. (Colors are visible in the online version of the article; <http://dx.doi.org/10.3233/AIC-140632>.)

4. Related work

The efficient delivery of Health Home Care service is increasingly attracting the attention of the AI and the Operations Research communities. Most of the contributions focus on the application aspects and often, beside the optimization approach, they describe the complete information system managing the service. Therefore the specific application aspects strongly characterize the optimization problems presented in each paper and the proposed solution methods. This is the reason why, in practice, each paper addresses a different problem and a thorough comparison can be hardly done.

The great majority of the contributions consider a service where patients/users specify a (more or less narrow) time window when they are available to receive the service. This feature is typical of the systems where the service is managed in conjunction with a Home Care. In Home Care services the presence of time windows is crucial since users are supposed to be independent and to move from their home. This is not the case of Home Health Care where we are in the presence of patients that, instead of being at the hospital, are in their bed at home and are not supposed to move.

One of the first papers dealing with this class of problems is [12]. The problem accounts for time windows and a mix of full time and part time workers. In the scheduling of full time workers a lunch break must be included and the objective is to minimize the cost of over time of full time workers and the time of part time workers. The authors propose a mathematical programming model and a simple heuristic.

Laps Care [14] is a system adopted in Sweden for Home Care, although it is also able to consider some of the issues in HHC. Laps Care is based on a set partitioning formulation of the problem and uses an iterative method. The starting solution proposes a single route for each service; then routes are joined until no further improvement is possible. To escape from the local optimum, one of the routes is split into one route for each patient, and the joining phase restarts. Another integrated system is presented in [3] where the focus is mainly on the GIS component of the Decision Support System. The problem is reduced to a vehicle routing and solved with classical heuristic methods.

A health home care system similar to ours is analyzed in [8]. One of the objectives is to balance the workload while some soft preference constraints for the nurse-patient assignment are accounted for as well as for the days of the week of the visits. Another objective is to minimize the number of uncovered visits that must be outsourced. The authors propose a simple assignment model ignoring the routing component, indeed the time of the visits is deterministically fixed to a constant time and this time includes the travel time. This assumption may be oversimplifying if we are in the presence of a wide territory to cover, in which the service and the travel times may vary from few minutes to one hour.

Rasmussen et al. [47] consider a specific core sub-problem that is the daily scheduling of nurses. In this problem the objective function accounts for many factors related mainly with the quality of service, such as the uncovered visits or a priority function, rather

than the operational cost which is not easily applicable on a single day. The problem is an extension of the classical Vehicle Routing Problem with Time Windows, widely studied in the literature. The authors propose a set partitioning formulation and an exact solution method based on Branch and Price that exploits a special branching rule taking advantage of the specific problem structure.

In [5], [13] and [57] authors address HHC problems with rather tight time windows. Bertels and Fahle [5] adopt an interesting hybrid approach, combining Constraint Programming, local search and Linear Programming. The approach takes advantage of the presence of narrow time windows: due to time window constraints, in the HHC only few permutations correspond to feasible orderings. The instance in Ferrara does not have such constraints, while retaining the loyalty and the workload balancing aspects, so the only criterion for assigning the order of patients in a nurse's schedule is minimizing the route. However, this does not mean that the problem is underconstrained or easier to solve. On the contrary, the search space is wider, as it is not pruned by simple constraints like time windows or nurse skills. The number of feasible solutions, on the other hand, does not necessarily increase: in a real-world instance new nurses are hired only when necessary, so the actual workload of the nurses is very close to their maximum working hours. This means that if the routes are not close to the optimum, the daily-workload constraint becomes infeasible: one has to find the optimum of the TSP just to find a feasible solution to the HHC problem or to assess infeasibility.

Another work that uses a hybrid CP-Mathematical programming approach is that in [42]. Here the authors consider the whole problem of assigning patients to nurses on a weekly base and make the daily schedule. The objective is twofold: on the one hand they minimize the overtime and the route length, on the other hand they account for a loyalty factor privileging the assignment of the same nurse to the same patient if possible. Also in this case time windows are present. The decomposition of the problem in a master weekly scheduling and a daily scheduling is also done in order to be able to adjust the service to last minute changes, which is very common in Health Home Care. A completely different approach to the last minute variability arising in request issues is that in [34] where a Markov process captures the stochastic aspects of the problem. Another stochastic model used in medium term planning is proposed in [36].

Looking at the problem from a more abstract viewpoint, one may see some similarities with the classi-

cal Capacitated Vehicle Routing Problem (CVRP). In the CVRP a set of disjoint routes for a fleet of vehicles has to be found so that all customers (nodes) are visited, the required quantity of goods is delivered to each customer, the capacity of the vehicles is not exceeded and the objective function is minimized. The usual objective function is the overall traveled distance or the number of vehicles. The CVRP has been tackled by many solution methods including branch and cut [2], metaheuristics [46], and hyperheuristics [18].

In our case we can see nurses as vehicles and patients as customers. There are some important differences with CVRP that make all the efficient method developed for the classical problem not applicable in our case. One difference concerns the capacity. As in CVRP, we may consider nurse daily duty time as a capacity constraint, however, unlike the CVRP, in our case the sequence in which patients are visited matters on how the capacity is consumed. This actually turns our problem into a time constrained VRP which is not as easy as the CVRP and for which the classical CVRP methods are not so efficiently adapted. The other difference concerns the objective function. On the one hand, as in VRP, we would have to minimize the total travelled distance, in order to make the service as efficient as possible, on the other hand we have to balance the workload among nurses. Thus this component of the objective function is a kind of bottleneck (min-max), that is difficult to address with OR methods. Finally, the loyalty is the component of the objective function that makes our problem very peculiar. The two criteria are often in contrast: any solution assigning to a single nurse all the services of the same patient would minimize the loyalty, but this would go to the detriment of workload balancing.

Large Neighborhood Search [44,53] is becoming more and more popular to solve routing problems. The idea is a local search that adopts a large neighborhood which makes less likely to fall in a local minimum. Exploring the neighborhood becomes a search problem itself, and it is done with CP. In this way, it is possible to take into account easily many side constraints.

The popularity of hybridization of CP and Local Search [41,62] is witnessed by the existence of solvers tailored for this approach [58], and the application to routing problems [32,49]. Various works deal with solving the TSP or its variant with CP or hybrid algorithms [10,16,43]. Rousseau et al. [49] propose a large neighborhood search in which CP explores a neighborhood with three operators. These operators are combined in variable neighborhood descent and a two

phase process. Kilby et al. [32] consider vehicle routing problems with side constraints, and compare classical OR approaches with CP models. OR approaches tend to use cost-oriented heuristics, that make them suitable for problems with few side constraints, but fail to find an initial solution when the number of side constraints grows. On the other hand, CP approaches are better when the side constraints are more tight but usually provide worse solutions when the side constraints are not prevalent.

5. Conclusions

We presented a real-life application of Constraint Solving technologies to the Home Health Care problem for the city of Ferrara, Italy.

The objectives, as requested by the chief nurses in charge of the service, were to provide schedules with balanced workloads without increasing working hours and travel times, and to improve the service for the patients avoiding the unpleasant situation in which a patient is taken care by many different nurses. These objectives were obtained through discussions with the chief nurses responsible for the service in Ferrara.

The results showed that the obtained solutions are significantly better under these respects than the ones previously obtained by hand by the nurses. The use of CP was considered a success; we decided to use it because of the ease of its use, and flexibility with respect to other techniques, such as Integer Programming. SAT and Answer Set Programming [9] are other interesting technologies to solve constrained optimization problems; modern solvers have sophisticated general-purpose search strategies, featuring restarts and no-good learning. CP, on the other hand, has powerful global constraints and allows the user to implement search strategies tailored for the given problem; in our experiments, custom search strategies had significantly better results than general-purpose ones; however it would be interesting to compare with a SAT or ASP solution.

A previous version of this work was also presented in the technical communication [11]. The current article extends [11] in several directions. First, the new implementation is in Comet, while in [11] it was in ECLⁱPS^e [52]. ECLⁱPS^e is a logic programming system, so in [11] we defined a declarative semantics for the traveltime constraint. On the other hand, Comet is more focussed on the efficiency, and contains more global constraints, so we were able, in this

paper, to propose a refined constraint model adopting the *multiknapsack* and *NValue* constraints, which can provide enhanced constraint propagation. Second, we used a new search algorithm, based on Large Neighbourhood Search, that proved to be the most effective for this problem. Third, we propose a deeper experimentation, with a set of crafted instances created from data of the real instances, and show the Pareto front of the two main objectives. Fourth, we studied a new objective function, based on the *deviation* constraint.

In the future, we plan to extend the decision span of the application, from weeks to months, as this could improve the loyalty further, possibly by exploiting as building block the developed procedure for the single week solution. It could also be interesting to explore different types of hybridization between CP, Local Search and Mixed Integer Linear Programming. Finally, considering that to reach a patient one has to spend some minimum time (corresponding to the closest patient location), some of the travel time could be moved to the service time, possibly improving the pruning of the *multiknapsack* constraint, in particular in instances with patients scattered in wide areas.

Acknowledgements

We would like to thank Dr. Caterina Montanari, Coordinatrice Assistenza Domiciliare Integrata (coordinator of the HHC program) of AUSL 109 of Ferrara, and Dr. Alessandra Galletti for explaining us the problem and providing the data of the instances.

We thank Andrea Peano for his help in the implementation.

References

- [1] D. Applegate, R.E. Bixby, V. Chvátal and W.J. Cook, Cutting planes and the traveling salesman problem (abstract only), in: *SODA*, D.B. Shmoys, ed., ACM/SIAM, 2000.
- [2] R. Baldacci, P. Toth and D. Vigo, Exact algorithms for routing problems under vehicle capacity constraints, *Annals of OR* **175** (2010), 213–245.
- [3] S.V. Begur, D.M. Miller and J.R. Weaver, An integrated spatial DSS for scheduling and routing home-health care nurses, *Interfaces* **27**(4) (1997), 35–48.
- [4] N. Beldiceanu and E. Contejean, Introducing global constraints in CHIP, *Mathematical and Computer Modelling* **20**(12) (1994), 97–123.
- [5] S. Bertels and T. Fahle, A hybrid setup for a hybrid scenario: combining heuristics for the home health care problem, *Computers & OR* **33**(10) (2006), 2866–2890.
- [6] C. Bessière, E. Hebrard, B. Hnich, Z. Kızıltan and T. Walsh, Filtering algorithms for the NValue constraint, *Constraints* **11**(4) (2006), 271–293.
- [7] C. Bessière, G. Katsirelos, N. Narodytska, C.-G. Quimper and T. Walsh, Decomposition of the NValue constraint, in: *CP*, D. Cohen, ed., Lecture Notes in Computer Science, Vol. 6308, Springer, 2010, pp. 114–128.
- [8] V. Borsani, A. Matta, G. Beschi and F. Sommaruga, A home care scheduling model for human resources, in: *International Conference on Service Systems and Service Management*, 2006, pp. 449–454.
- [9] G. Brewka, T. Eiter and M. Truszczyński, Answer set programming at a glance, *Commun. ACM* **54**(12) (2011), 92–103.
- [10] Y. Caseau and F. Laburthe, Solving small TSPs with constraints, in: *Proc. of the 14th International Conference on Logic Programming*, L. Naish, ed., 1997.
- [11] M. Cattafi, R. Herrero, M. Gavanelli, M. Nonato and F. Malucelli, Improving quality and efficiency in home health care: An application of constraint logic programming for the Ferrara NHS unit, in: *ICLP (Technical Communications)*, A. Dovier and V. Santos Costa, eds, LIPIcs, Vol. 17, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2012, pp. 415–424.
- [12] E. Cheng and J.L. Rich, A home health care routing and scheduling problem, Technical Report CAAM TR98–04, Rice University, 1998.
- [13] B. Elbenani, J.A. Ferland and V. Gascon, Mathematical programming approach for routing home care nurses, in: *IEEE International Conference on, Industrial Engineering and Engineering Management, IEEM 2008*, 2008, pp. 107–111.
- [14] P. Eveborn, M. Rönnqvist, H. Einarssdóttir, M. Eklund, K. Lidén and M. Almroth, Operations research improves quality and efficiency in home care, *Interfaces* **39** (2009), 18–34.
- [15] M.L. Fisher, The Lagrangian relaxation method for solving integer programming problems, *Management Science* **27** (1981), 1–18.
- [16] F. Focacci, A. Lodi and M. Milano, A hybrid exact algorithm for the TSPTW, *INFORMS Journal on Computing* **14**(4) (2002), 403–417.
- [17] A.S. Fukunaga and R.E. Korf, Bin completion algorithms for multicontainer packing, knapsack, and covering problems, *J. Artif. Int. Res.* **28** (2007), 393–429.
- [18] P. Garrido, C. Castro and E. Monfroy, Towards a flexible and adaptable hyperheuristic approach for VRPs, in: *IC-AI*, H. Arabnia, D. de la Fuente and J. Olivas, eds, 2009.
- [19] M. Gavanelli, An algorithm for multi-criteria optimization in CSPs, in: *ECAI 2002*, IOS Press, 2002.
- [20] M. Gebser, R. Kaminski, B. Kaufmann, M. Ostrowski, T. Schaub and M. Schneider, Potassco: The Potsdam answer set solving collection, *AI Communications* **24**(2) (2011), 105–124.
- [21] M.V. Giuliani, M. Scopelliti and F. Fornara, Elderly people at home: Technological help in everyday activities, in: *IEEE International Workshop on Robot and Human Interactive Communication, ROMAN 2005*, 2005, pp. 365–370.
- [22] J.L. Goffin, On convergence rates of subgradient optimization methods, *Mathematical Programming* **13** (1977), 329–347.
- [23] C.P. Gomes, B. Selman and N. Crato, Heavy-tailed distributions in combinatorial search, in: *CP*, G. Smolka, ed., Lecture Notes in Computer Science, Vol. 1330, Springer, 1997, pp. 121–135.

- [24] D. Guimarans, R. Herrero, J.J. Ramos and S. Padrón, Solving vehicle routing problems using constraint programming and Lagrangian relaxation in a metaheuristics framework, in: *Management Innovations for Intelligent Supply Chains*, IGI Global, 2013, pp. 123–143.
- [25] D. Guimarans, R. Herrero, D. Riera, A.A. Juan and J.J. Ramos, Combining probabilistic algorithms, constraint programming and Lagrangian relaxation to solve the vehicle routing problem, *Ann. Math. Artif. Intell.* **62**(3,4) (2011), 299–315.
- [26] R.M. Haralick and G.L. Elliott, Increasing tree search efficiency for constraint satisfaction problems, *Artif. Intell.* **14**(3) (1980), 263–313.
- [27] E. Hebrard, D. Marx, B. O’Sullivan and I. Razgon, Soft constraints of difference and equality, *J. Artif. Intell. Res. (JAIR)* **41** (2011), 97–130.
- [28] M. Held and R.M. Karp, The travelling salesman problem and minimum spanning trees: Part II, *Mathematical Programming* **1** (1971), 6–25.
- [29] T. Hulubei and B. O’Sullivan, The impact of search heuristics on heavy-tailed behaviour, *Constraints* **11**(2,3) (2006), 159–178.
- [30] H. Kautz, E. Horvitz, Y. Ruan, C. Gomes and B. Selman, Dynamic restart policies, in: *AAAI/IAAI*, 2002.
- [31] L.G. Kaya and J.N. Hooker, A filter for the circuit constraint, in: *CP*, F. Benhamou, ed., Lecture Notes in Computer Science, Vol. 4204, Springer, 2006, pp. 706–710.
- [32] P. Kilby, P. Prosser and P. Shaw, A comparison of traditional and constraint-based heuristic methods on VRPs with side constraints, *Constraints* **5** (2000), 389–414.
- [33] D.E. Knuth, Two notes on notation, *Am. Math. Monthly* **99** (1992), 403–422.
- [34] P.M. Koeleman, S. Bhulai and M. van Meersbergen, Optimal patient and personnel scheduling policies for care-at-home service facilities, *European Journal of Operational Research* **219**(3) (2012), 557–563.
- [35] R.E. Korf, An improved algorithm for optimal bin packing, in: *IJCAI*, G. Gottlob and T. Walsh, eds, Morgan Kaufmann, 2003, pp. 1252–1258.
- [36] E. Lanzarone, A. Matta and G. Scaccabarozzi, A patient stochastic model to support human resource planning in home care, *Production Planning and Control* **21**(1) (2010), 3–25.
- [37] M. Luby, A. Sinclair and D. Zuckerman, Optimal speedup of Las Vegas algorithms, *Inf. Process. Lett.* **47** (1993), 173–180.
- [38] P. Meseguer and C. Torras, Exploiting symmetries within constraint satisfaction search, *Artif. Intell.* **129**(1,2) (2001), 133–163.
- [39] A. Mileo, D. Merico and R. Bisiani, Reasoning support for risk prediction and prevention in independent living, *Theory and Practice of Logic Programming* **11**(2,3) (2011), 361–395.
- [40] J.N. Monette, P. Schaus, S. Zampelli, Y. Deville and P. Dupont, A CP approach to the balanced academic curriculum problem, in: *SymCon*, B. Benhamou, B. Choueiry and B. Hnich, eds, 2007.
- [41] E. Monfroy, F. Saubion and T. Lambert, On hybridization of local search and constraint propagation, in: *ICLP*, B. Demoen and V. Lifschitz, eds, 2004.
- [42] S. Nickel, M. Schröder and J. Steeg, Mid-term and short-term planning support for home health care services, *European Journal of Operational Research* **219**(3) (2012), 574–587.
- [43] G. Pesant, M. Gendreau, J.-Y. Potvin and J.-M. Rousseau, An exact constraint logic programming algorithm for the TSP with time windows, *Transp. Sci.* **32** (1998), 12–29.
- [44] G. Pesant, M. Gendreau and J.-M. Rousseau, GENIUS-CP: A generic single-vehicle routing algorithm, in: *CP’97*, Springer, 1997.
- [45] G. Pesant and J.-C.R. Spread, A balancing constraint based on statistics, in: *CP*, P. van Beek, ed., Lecture Notes in Computer Science, Vol. 3709, Springer, 2005.
- [46] D. Pisinger and S. Ropke, A general heuristic for vehicle routing problems, *Computers & Operations Research* **34**(8) (2007), 2403–2435.
- [47] M.S. Rasmussen, T. Justesen, A. Dohn and J. Larsen, The home care crew scheduling problem: Preference-based visit clustering and temporal dependencies, *European Journal of Operational Research* **219** (2012), 598–610.
- [48] J.-C. Régim, A filtering algorithm for constraints of difference in CSPs, in: *AAAI*, B. Hayes-Roth and R.E. Korf, eds, AAAI Press/The MIT Press, 1994, pp. 362–367.
- [49] L.-M. Rousseau, M. Gendreau and G. Pesant, Using constraint-based operators to solve the vehicle routing problem with time windows, *J. Heuristics* **8** (2002), 43–58.
- [50] P. Schaus, Y. Deville, P. Dupont and J.-C. Régim, The deviation constraint, in: *CPAIOR*, W. van Hoeve and L.A. Wolsey, eds, LNCS, Vol. 4510, Springer, 2007.
- [51] P. Schaus, P. Van Hentenryck and J.-C. Régim, Scalable load balancing in nurse to patient assignment problems, in: *CPAIOR*, W.J. van Hoeve and J.N. Hooker, eds, Lecture Notes in Computer Science, Vol. 5547, Springer, 2009, pp. 248–262.
- [52] J. Schimpf and K. Shen, Ecl¹ps^c – from LP to CLP, *Theory and Practice of Logic Programming* **12**(1,2) (2012), 127–156.
- [53] P. Shaw, Using constraint programming and local search methods to solve vehicle routing problems, in: *CP98*, M.J. Maher and J.-F. Puget, eds, LNCS, Vol. 1520, 1998.
- [54] P. Shaw, A constraint for bin packing, in: *Proceedings, Principles and Practice of Constraint Programming – CP 2004, 10th International Conference*, Toronto, Canada, 27 September–1 October 2004, M. Wallace, ed., LNCS, Vol. 3258, Springer, 2004, pp. 648–662.
- [55] H. Simonis, Models for global constraint applications, *Constraints* **12** (2007), 63–92.
- [56] C. Sinz and M. Iser, Problem-sensitive restart heuristics for the DPLL procedure, in: *SAT*, O. Kullmann, ed., LNCS, Vol. 5584, Springer, 2009.
- [57] J. Steeg and M. Schröder, A hybrid approach to solve the periodic home health care problem, in: *Operations Research Proceedings 2007*, J. Kalcsics and S. Nickel, eds, Vol. 2007, Springer, Berlin/Heidelberg, 2008, pp. 297–302.
- [58] P. Van Hentenryck and L. Michel, *Constraint-Based Local Search*, MIT Press, 2005.
- [59] W.J. van Hoeve, The alldifferent constraint: A survey, in: *Proceedings of the 6th Annual Workshop of the ERCIM Working Group on Constraints*, K.R. Apt, R. Barták, E. Monfroy and F. Rossi, eds, CoRR, Vol. Cs.PL/0105015, 2001.
- [60] W.J. van Hoeve, A hyper-arc consistency algorithm for the soft alldifferent constraint, in: *Proceedings, Principles and Practice of Constraint Programming – CP 2004, 10th International Conference*, Toronto, Canada, 27 September–1 October 2004, M. Wallace, ed., LNCS, Vol. 3258, Springer, 2004, pp. 679–689.

- [61] L.N. Van Wassenhove and L.F. Gelders, Solving a bicriterion scheduling problem, *European Journal of Operational Research* **4**(1) (1980), 42–48.
- [62] M. Wallace, Hybrid algorithms in constraint programming, in: *CSCLP*, F. Azevedo, P. Barahona, F. Fages and F. Rossi, eds, LNCS, Vol. 4651, Springer, 2006.

Copyright of AI Communications is the property of IOS Press and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.